

University of Hertfordshire

Hosted by Global Academic Foundation

**School of Engineering and Computer Science
Mechanical Engineering and Mechatronics Program**

Digital Electronics (4FTC2027)

Project Title

Smart Room Temperature Control

Group Members

- Youssef Ahmed Galal**
- Ahmed Osama Ibrahim**
- Amy Ahmed Reda**
- Salsabeel Ahmed Abdelhameed**

Module Leader

Dr. Samar Moustafa

Submission Date: 18/12/2025

TABLE OF CONTENTS

1. INTRODUCTION	1
2. PROPOSED SYSTEM FEATURES	2
3. SYSTEM DESIGN AND FLOW CHARTS	3
4. HARDWARE COMPONENTS.....	4
5. CIRCUIT DIAGRAM	5
6. SOFTWARE DESIGN	6
7. TESTING AND RESULTS	7
8. CHALLENGES & SOLUTIONS.....	8
9. FUTURE IMPROVEMENTS.....	9
10. CONCLUSION	10
11. REFERENCES	11
12. APPENDIX.....	12

1. Introduction

This project is about the smart room temperature control, which aims to monitor the temperature of the room and control a fan to provide comfortable conditions. This system is beneficial and commonly used in real life. For example, the system allows for stable control of the temperature in the room, while also reducing energy consumption. The systems used in real life also provide remote control of the system as well as the ability to set custom temperature schedules. These systems can also be integrated with the other smart home systems in the house. The system that we built activated the fan by two ways: automatically (using Sensor), and manually (using Buttons), and this can be toggled by user.

2. Proposed System Features

Our project consists of many features:

- Providing automatic mode operation:

Using the auto button, the auto mode starts, and the fan speed is determined by the temperature reading from the sensor, where the temperature exceeds a certain threshold (25), fan speed increases (Low).

- Providing Manual mode operation:

Using the same auto button to change the mode from auto to manual. This mode mainly depends on physical intervention; we add another 4 buttons which represent the following fan modes (Off, Low, Medium, and High), which only work in the manual mode.

- LCD:

The LCD added in our project shows the following: Temperature, Humidity, Fan speed (High, Medium, Low, Off), and the mode which is Automatic (AU), or Manual (MA).

The customer can turn it on and leave it to enable or disable fan speed according to the temperature or choose their own fan speed using the buttons available and all is displayed on LCD for easier understanding of system.

3. System Design and Flow Charts

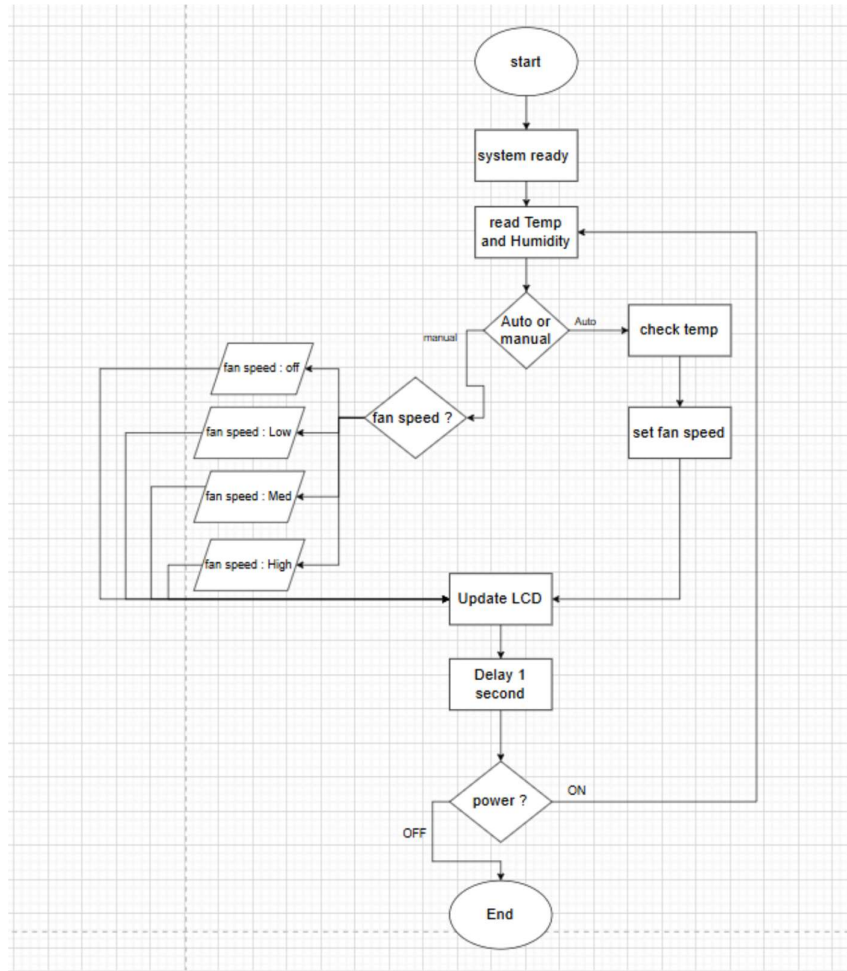


Figure 1: Flowchart for System Design

The flow chart above describes the operation of the fan control system in terms of its order of operation. The first stage is system boot-up. During this stage, temperature and humidity information is received by the sensor. The system then determines what Operating Mode has been selected by the user. The Operating Mode can either be the Automatic Operating Mode (the fan speeds are calculated based on the measured temperature) or the Manual Operating Mode (the fan speed is selected by the user (off, low, medium, high)). The LCD displays both the selected fan speed and temperature and humidity data from the sensor. There is a one-second delay to allow the fan to stabilise at the selected speed after activation. While the system is powered ON, it will continue to perform this cycle repeatedly until the power is turned OFF.

4. Hardware Components

- **Arduino Uno:** is the system's main controller that is responsible for reading sensor data and managing system's outputs accordingly.
- **DHT22 temperature and humidity sensor:** detects the temperature and humidity inside a room accurately.
- **16x2 I2C LCD display:** displays the temperature, humidity, and status of the fan.
- **H-bridge driver:** connects and provides sufficient current for the fan.
- **5V fan:** is the cooling system of the setup that turns ON or OFF based on temperature.
- **Push buttons:** enable user input, which changes the mode of operation and fan speeds.
- **4.7k Ω Resistor:** is used to ensure a stable digital signal for the sensor's data pin.
- **Battery cells and battery holder:** are used to power the fan and the motor driver.
- **Breadboard and jumper wires:** are used to assemble the circuit.

Part name	Quantity	Cost (egp)
Arduino Uno	1	340
DHT22 Sensor	1	285
16x2 I2C LCD display	1	95
L298N H bridge	1	80
5V fan	1	60
Push buttons	5	1
Resistor	1	0.5
Battery cells	3	45
Battery holder	1	20
Breadboard	1	40
Jumper Wires		1.5
Total Cost		1,062

5. Circuit Diagram

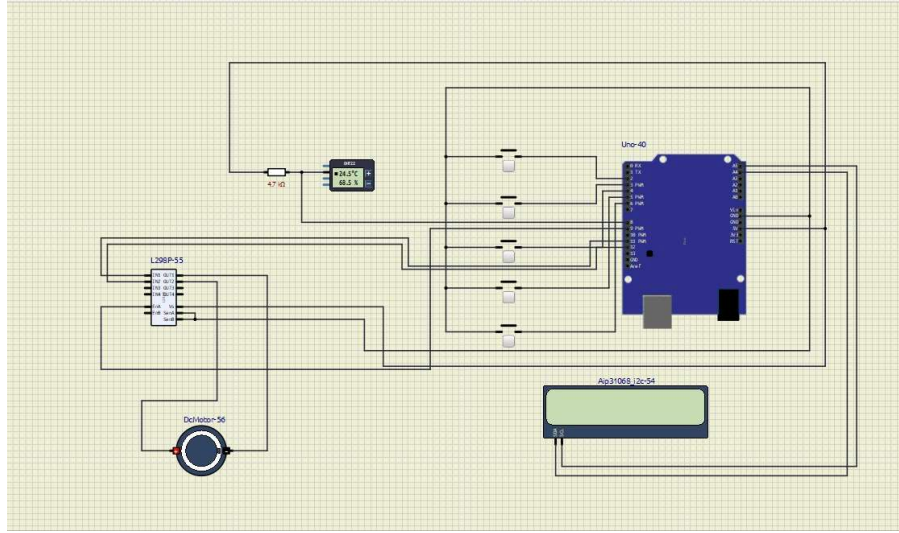


Figure 2: Circuit Design in SimulIDE

The figure above presents the complete circuit schematic of the smart room temperature monitoring and control system in SimulIDE.

- **The DHT22 temperature and humidity sensor** is connected to D8 on the Arduino board. The VCC and GND pins of the sensor are connected to the 5V and GND pins, respectively. A 4.7kΩ resistor is used as a pull up resistor on the data pin.
- **The LCD module** is connected to the Arduino through the I2C communication protocol. The SDA and SCL pins of the LCD module are connected to the Arduino's SDA and SCL pins (A4 and A5 respectively).
- **The five push buttons** are connected to the Arduino as follows: OFF button is connected to D3, low mode button to D4, medium mode button to D5, high mode button to D6, auto/manual mode button to D2. Every button is connected with the GND pin.
- **The L298N H-Bridge motor driver module** control inputs are linked to the Arduino board as follows: pin IN3 to D11, pin IN4 to D12, and the enable pin ENB to D9 (in SimulIDE, IN1, IN2, and ENA are the pins connected instead). The 5V pin in the motor driver is connected to 5V of the Arduino, and the three battery cells power the motor driver by connecting them to the 12V and GND of the motor driver.
- **The 5V fan** is connected to motor driver through the OUT3 and OUT4 pins (connected to OUT1 and OUT2 in SimulIDE)
- Each component is assembled on a breadboard with the help of jumper wire

6. Software Design

For this project, object oriented programming (OOP) was used, which means there was a tab for each component's code, and then the main tab was the code that grouped them all together. There were four different components that needed codes: LCD, DHT Sensor, Push Buttons, and Motor Driver, and there were two tabs for hardware (definitions and pin assignment) and functions (name of functions so the main tab can read them).

- First, the hardware and functions tab were included, and variables were declared (isAuto and fanMode). In the main setup, all components were setup. The main loop continuously updates every second. The loop starts by reading temperature and humidity from sensor, then the code checks whether the isAuto variable is true or false. If true, then automatic control of the fan is activated, and the fan speed is decided by temperature. If false, then manual control of the fan is activated, and the fan speed is decided by the 4 push buttons. The LCD is updated to show the temperature, humidity, fan speed, and auto (AU)/manual (MA), and this whole loop happens every second.
- For the component's code, the LCD and DHT Sensor were brought from the library with codes named LiquidCrystal_I2C and DHT respectively. The LCD code was setup to display System Ready at the start before the loop, then show the variables mentioned before. The DHT Sensor code was setup to read temperature and humidity and return a value of -1 if there was an error with the sensor.
- The buttons code had two different functions: one for the speeds (4 buttons) and one for the auto and manual toggle (1 button). The speed function (called fanSpeedBtns) depended on a variable called currentMode, which was the same as the fanMode variable. In this function, if a button was pressed (equals LOW as the buttons were connected as input pullups), it will return the value of corresponding button. If no button was pressed it will return the same value as previous loop. For example, if in a loop the medium speed button was pressed (BTN_MED), the value returned for the fanMode would be 2. In the next loop, if no button is pressed, the value would remain 2. The second function was for auto/manual control of the fan, and depends on the isAuto variable, where if the button is pressed in the loop, and was not pressed in the previous loop, the isAuto variable will switch. Other than that, no change occurs.
- For the motor driver, the fanMode determined the speed of the fan. For example, if fanMode was set to 2, then the speed of the motor driver would be medium. The code also included automatic control, which returns a fanMode according to temperature, then the function mentioned above was executed. This was also the same for the manual control.

7. Testing and Results

After finishing the code and verifying it in Arduino IDE to make sure there were no mistakes, the code was placed first in a simulation to check if it was correct. After making sure all the connections in SimulIDE were correct, the system was tested, and it worked as exactly as expected. Then the same connections were made on the breadboard. The system was working, with the LCD showing the values as expected, but the LOW fan speed didn't make the fan rotate, while the other two speeds (MED and HIGH) were working well. So, after changing the numbers in the codes for the fan speed and testing again, the system worked well and as intended.

8. Challenges & Solutions

One of the challenges we faced was the lifetime of the batteries used as every mode worked well when the batteries were fully charged as shown in the video, but after it lost some charge, the low speed wasn't working. The solution for this was to recharge batteries, as these batteries were rechargeable, but unfortunately, we didn't have time to implement a way to do that. Another issue we encountered was that the low fan mode wasn't working at the beginning, as the fan needed more voltage to start. Therefore, instead of having each mode at exactly third of the maximum voltage, the differences between each mode were slightly decreased, but the low fan mode now had a larger voltage to go to the fan. Finally, there was a challenge in the start of testing in real life, which was that the LCD wasn't working. This turned out to be because the code had the wrong LCD address coded, as the LCD in SimulIDE had a different address compared to the LCD given to us in real life.

9. Future Improvements

The system can be improved by allowing remote control through a mobile application. Sensor accuracy and response time can also be improved to enhance the system. This could be done by adding a more accurate sensor than the DHT22. Additionally, adding an option for the user to choose the temperature threshold that the fan speed will be based on. Finally, energy used by the system could be displayed to improve energy efficiency by the system.

10. Conclusion

The goal of the project was to create a system to monitor temperature and according to that temperature, the fan speed changes or completely shuts off. We managed to provide 2 different options: First, the Automatic mode operation, which turns on or off fan and changes its speed according to certain temperature thresholds. The second option is the Manual mode operation, which the user chooses the fan to operate at a certain speed. The project worked well in both the simulation and the hardware according to our expectations and the short time that we had, and there were relatively few issues that we had to overcome. If more time was available, we could have tried to implement the remote-control system, as well as found an idea to close the casing and allow push buttons to be at the top of the casing.

11. References

Adafruit (no date) *GitHub - adafruit/DHT-sensor-library: Arduino library for DHT11, DHT22, etc Temperature & Humidity Sensors.*
<https://github.com/adafruit/DHT-sensor-library>. (Accessed: 18/12/2025)

Aosong Electronics Co.,Ltd and Liu, T. (no date) *Digital-output relative humidity & temperature sensor/module DHT22 (DHT22 also named as AM2302).*
<https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf>. (Accessed: 18/12/2025)

Future Electronics Egypt (Arduino Egypt) (no date). https://store.futureelectronics.com/?srsId=AfmBOooVMCGh5fs3cGqxBgqWzeA85tc_KHB8JhIbu4tHjNsHEdy-pss. (Accessed: 18/12/2025)

Handson Technology (no date) *User Guide.*
https://components101.com/sites/default/files/component_datasheet/L298N-Motor-Driver-Datasheet.pdf. (Accessed: 18/12/2025)

Johnrickman (no date) *GitHub - johnrickman/LiquidCrystal_I2C: LiquidCrystal Arduino library for the DFRobot I2C LCD displays.*
https://github.com/johnrickman/LiquidCrystal_I2C?tab=readme-ov-file. (Accessed: 18/12/2025)

PHLIPS (no date) *I2C LCD1602 Module, www.sunfounder.com.*
<https://www.sunfounder.com>. (Accessed: 18/12/2025)

12. Appendix

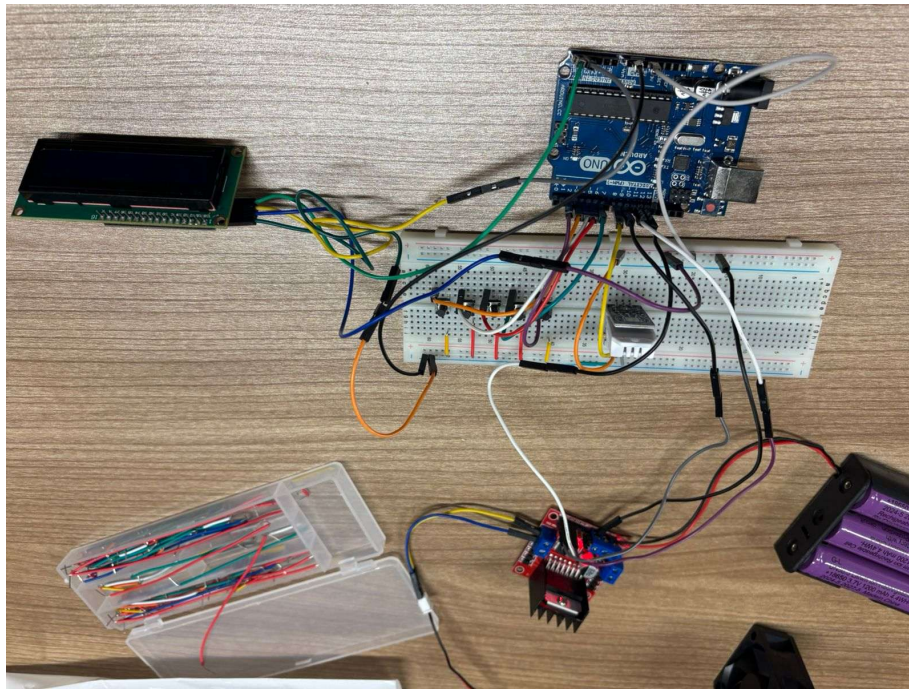


Figure 3: Real Circuit Design

```
digitalProjectTemperatureControlMonitor.ino Buttons.ino DHTSensor.ino LCD.ino MotorD
1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3
4 LiquidCrystal_I2C lcd(0x27, 16, 2);
5
6 bool lcdPower = true;
7
8
9 void setupLCD() {
10   lcd.init();
11   lcd.clear();
12   lcd.backlight();
13   lcd.setCursor(0,0);
14   lcd.print("System Ready");
15   delay(1000);
16 }
17
18 void updateLCD(float temp, float hum, int fanMode, bool isAuto) {
19
20   if (!lcdPower) return;
21
22   lcd.clear();
23   lcd.setCursor(0,0);
24   lcd.print("T:");
25   lcd.print(temp, 1);
26   lcd.print("C ");
27
28   lcd.print("H:");
29   lcd.print(hum);
30   lcd.print("%");
31
32   lcd.setCursor(0,1);
33   lcd.print ("Fan Speed:");
34   if (fanMode == 0) {
35     lcd.print("OFF");
36   } else if (fanMode == 1) {
37     lcd.print ("LOW");
38   } else if (fanMode == 2) {
39     lcd.print ("MED");
40   } else if (fanMode == 3) {
41     lcd.print ("HIGH");
42   }
43   if (isAuto) {
44     lcd.print (" AU");
45   } else if (!isAuto) {
46     lcd.print (" MA");
47   }
48 }
```

Figure 4: LCD Code

```
digitalProjectTemperatureControlMonitor.ino Buttons.ino DHTSensor.ino
1 #include "hardware.h"
2 #include <DHT.h>
3
4 DHT dht(DHT_PIN, DHTTYPE);
5
6 void setupDHT(){
7   dht.begin();
8 }
9
10 float readTemperature(){
11   float temp = dht.readTemperature();
12
13   if (isnan(temp)) {
14     return -1;
15   }
16
17   return temp;
18 }
19
20 float readHumidity() {
21   float hum = dht.readHumidity();
22
23   if (isnan(hum)) {
24     return -1;
25   }
26   return hum;
27 }
```

Figure 5: Sensor Code

```

digitalProjectTemperatureControlMonitor.ino Buttons.ino DHTSe
1 #include "hardware.h"
2
3 void setupBtns () {
4     pinMode (BTN_OFF , INPUT_PULLUP);
5     pinMode (BTN_LOW , INPUT_PULLUP);
6     pinMode (BTN_MED , INPUT_PULLUP);
7     pinMode (BTN_HIGH , INPUT_PULLUP);
8     pinMode (BTN_AUTO , INPUT_PULLUP);
9 }
10
11
12 int fanSpeedBtns (int currentMode) {
13     if (digitalRead(BTN_OFF) == LOW) {
14         return 0;
15     }
16     if (digitalRead(BTN_LOW) == LOW) {
17         return 1;
18     }
19     if (digitalRead(BTN_MED) == LOW) {
20         return 2;
21     }
22     if (digitalRead(BTN_HIGH) == LOW) {
23         return 3;
24     }
25     return currentMode;
26 }
27
28 bool toggleAuto (bool currentState) {
29     static bool lastState = HIGH;
30     bool state = digitalRead (BTN_AUTO);
31     if (state == LOW && lastState == HIGH) {
32         currentState = !currentState;
33     }
34     lastState = state;
35     return currentState;
36 }

```

Figure 6: Buttons Code

```

digitalProjectTemperatureControlMonitor.ino Buttons.ino DHTSensor.ino LCD.ino MotorDriver.ino
1 #include "hardware.h"
2
3 void setupMotor () {
4     pinMode (IN3, OUTPUT);
5     pinMode (IN4, OUTPUT);
6     pinMode (ENB, OUTPUT);
7 }
8
9 void fanDirection () {
10     digitalWrite (IN3, HIGH);
11     digitalWrite (IN4, LOW);
12 }
13
14 void applyFanMode () {
15     fanDirection ();
16     if (fanMode == 0) {
17         analogWrite(ENB, 0);
18     } else if (fanMode == 1) {
19         analogWrite(ENB, 150);
20     } else if (fanMode == 2) {
21         analogWrite(ENB, 190);
22     } else if (fanMode == 3) {
23         analogWrite(ENB, 255);
24     }
25 }
26
27 void autoControl (float temp) {
28     if (temp <= tempThreshold1) {
29         fanMode = 0;
30     }
31     else if (temp > tempThreshold1 && temp <= tempThreshold2) {
32         fanMode = 1;
33     }
34     else if (temp > tempThreshold2 && temp <= tempThreshold3) {
35         fanMode = 2;
36     }
37     else if (temp > tempThreshold3) {
38         fanMode = 3;
39     }
40     applyFanMode ();
41 }
42
43 void manualControl () {
44     fanMode = fanSpeedBtns (fanMode);
45     applyFanMode ();
46 }

```

Figure 7: Motor Driver Code

```

digitalProjectTemperatureControlMonitor.ino Buttons.ino DHTSensor.ino LCD.ino MotorDriver.ino functions.h
1 #pragma once
2
3 void setupBtns ();
4 int fanSpeedBtns (int currentMode);
5 bool toggleAuto (bool currentState);
6
7 void setupLCD ();
8 void updateLCD (float temp, float hum, int fanMode, bool isAuto);
9
10 void setupDHT ();
11 float readTemperature ();
12 float readHumidity ();
13
14 void setupMotor ();
15 void fanDirection ();
16 void applyFanMode ();
17 void autoControl (float temp);
18 void manualControl ();
19

```

Figure 8: Functions Code

```

digitalProjectTemperatureControlMonitor.ino Buttons.ino
1 #pragma once
2
3 #define BTN_OFF 3
4 #define BTN_LOW 4
5 #define BTN_MED 5
6 #define BTN_HIGH 6
7 #define BTN_AUTO 2
8 #define DHT_PIN 8
9 #define DHTTYPE DHT22
10 #define IN3 11
11 #define IN4 12
12 #define ENB 9
13 #define tempThreshold1 25
14 #define tempThreshold2 30
15 #define tempThreshold3 35
16
17 extern int fanMode;

```

Figure 9: Hardware Code

```
digitalProjectTemperatureControlMonitor.ino  Buttons.ino  DHT
1  #include "hardware.h"
2  #include "functions.h"
3
4  int fanMode = 0;
5  bool isAuto = true;
6
7  void setup() {
8      setupBtns ();
9      setupDHT ();
10     setupLCD ();
11     setupMotor ();
12     fanDirection ();
13 }
14
15 void loop() {
16     float temp = readTemperature ();
17     float hum = readHumidity ();
18     isAuto = toggleAuto(isAuto);
19
20     if (isAuto) {
21         autoControl (temp);
22     } else if (!isAuto) {
23         manualControl ();
24     }
25
26     updateLCD(temp, hum, fanMode, isAuto);
27     delay (1000);
28 }
29
```

Figure 10: Main Code